

MÓDULO 5: **IMPLEMENTACIÓN SQL Y BASE DE DATOS**

Proyecto: Análisis ETL - Entidad Financiera

Prerequisito: MÓDULO 4 completado (análisis y visualizaciones listas)

OBJETIVO

Implementar capa de persistencia con SQL:

- Diseñar esquema de base de datos relacional
- Crear base de datos SQLite
- Cargar datos limpios a tablas
- Ejecutar consultas analíticas SQL
- Integrar SQL con Python

PASO 1: DISEÑAR ESQUEMA DE BASE DE DATOS

► 1.1 Análisis de entidades

Entidades identificadas:

Entidad	Descripción	Clave Primaria
Customers	Información de clientes	<code>customer_id</code>
Sales	Transacciones de ventas	<code>transaction_id</code>
Products	Categorías de productos	<code>product_category</code>
Locations	Ubicaciones de tiendas	<code>location</code>

Relaciones:

- Customers 1:N Sales (un cliente puede tener muchas ventas)
- Products 1:N Sales (una categoría tiene muchas ventas)
- Locations 1:N Sales (una ubicación tiene muchas ventas)

► 1.2 Crear archivo schema.sql

En VS Code:

1. Crear carpeta `sql/` (si no existe)
2. Crear archivo `sql/schema.sql`
3. Escribir el siguiente código:

Archivo: `sql/schema.sql`

```
-- =====
-- ESQUEMA DE BASE DE DATOS: ANÁLISIS FINANCIERO
-- Proyecto: ETL Entidad Financiera
-- Fecha: 8 de noviembre de 2025
-- =====

-- Eliminar tablas si existen (para re-ejecución)
DROP TABLE IF EXISTS sales;
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS locations;

-- =====
-- TABLA: customers
-- Descripción: Información de clientes de la entidad financiera
-- =====
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    age REAL NOT NULL,
    gender TEXT NOT NULL CHECK(gender IN ('Male', 'Female')),
    location TEXT NOT NULL,
    membership_years INTEGER NOT NULL,
    CONSTRAINT chk_age CHECK(age >= 18 AND age <= 100),
    CONSTRAINT chk_membership CHECK(membership_years >= 0)
);

-- Índices para optimizar búsquedas
CREATE INDEX idx_customers_gender ON customers(gender);
CREATE INDEX idx_customers_location ON customers(location);
CREATE INDEX idx_customers_age ON customers(age);

-- =====
-- TABLA: products
-- Descripción: Catálogo de categorías de productos
-- =====
CREATE TABLE products (
    product_category TEXT PRIMARY KEY,
    description TEXT
);

-- =====
-- TABLA: locations
-- Descripción: Ubicaciones de centros comerciales
-- =====
CREATE TABLE locations (
```

```
location TEXT PRIMARY KEY,
city TEXT NOT NULL,
country TEXT DEFAULT 'Turkey'
);

-- =====
-- TABLA: sales
-- Descripción: Registro de transacciones de venta
-- =====
CREATE TABLE sales (
    transaction_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    invoice_date DATE NOT NULL,
    invoice_no TEXT NOT NULL,
    product_category TEXT NOT NULL,
    quantity INTEGER NOT NULL CHECK(quantity > 0),
    price REAL NOT NULL CHECK(price > 0),
    payment_method TEXT NOT NULL CHECK(payment_method IN ('Cash', 'Credit Card', 'Debit Card')),
    shopping_mall TEXT NOT NULL,
    total_sale REAL GENERATED ALWAYS AS (quantity * price) STORED,

    -- Claves foráneas
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE CASCADE,
    FOREIGN KEY (product_category) REFERENCES products(product_category) ON DELETE RESTRICT
);

-- Índices para optimizar consultas
CREATE INDEX idx_sales_customer ON sales(customer_id);
CREATE INDEX idx_sales_date ON sales(invoice_date);
CREATE INDEX idx_sales_category ON sales(product_category);
CREATE INDEX idx_sales_payment ON sales(payment_method);
CREATE INDEX idx_sales_mall ON sales(shopping_mall);

-- =====
-- VISTA: Ventas con información de clientes
-- =====
CREATE VIEW vw_sales_complete AS
SELECT
    s.transaction_id,
    s.customer_id,
    c.age,
    c.gender,
    c.location,
    c.membership_years,
    s.invoice_date,
    s.product_category,
```

```
s.quantity,
s.price,
s.total_sale,
s.payment_method,
s.shopping_mall
FROM sales s
INNER JOIN customers c ON s.customer_id = c.customer_id;

-- =====
-- FIN DEL ESQUEMA
-- =====
```

Elementos SQL clave:

Elemento	Descripción
PRIMARY KEY	Identifica únicamente cada registro
FOREIGN KEY	Relación con otra tabla
NOT NULL	Campo obligatorio
CHECK	Validación de valores
INDEX	Mejora rendimiento de búsquedas
GENERATED ALWAYS AS	Columna calculada automáticamente
VIEW	Consulta guardada como tabla virtual

 PASO 2: CREAR BASE DE DATOS SQLITE

► 2.1 Importar biblioteca SQLite3

Nueva celda en notebook:

```
import sqlite3
import os

print('=' * 80)
print('CREACIÓN DE BASE DE DATOS SQLITE')
print('=' * 80)

# Ruta de la base de datos
db_path = '../sql/financial_analysis.db'

# Eliminar base de datos existente (para limpieza)
if os.path.exists(db_path):
    os.remove(db_path)
    print(f'✅ Base de datos anterior eliminada')

# Crear nueva base de datos
conn = sqlite3.connect(db_path)
cursor = conn.cursor()

print(f'✅ Base de datos creada: {db_path}')
print(f'📊 Conexión establecida')
```

¿Qué hace SQLite3?

- Biblioteca estándar de Python (no requiere pip install)
- Base de datos embebida (archivo .db)
- No requiere servidor separado
- Ideal para análisis de datos

Comandos SQLite3:

Comando	Función
<code>sqlite3.connect('archivo.db')</code>	Crear/abrir base de datos
<code>conn.cursor()</code>	Crear cursor para ejecutar SQL
<code>cursor.execute(sql)</code>	Ejecutar sentencia SQL
<code>cursor.executemany(sql, data)</code>	Ejecutar múltiples inserciones
<code>conn.commit()</code>	Guardar cambios
<code>conn.close()</code>	Cerrar conexión

Salida:

```
=====
CREACIÓN DE BASE DE DATOS SQLITE
=====
[✓] Base de datos anterior eliminada
[✓] Base de datos creada: ../sql/financial_analysis.db
[📊] Conexión establecida
```

► 2.2 Ejecutar schema.sql**Nueva celda:**

```
# Leer archivo schema.sql
with open('../sql/schema.sql', 'r', encoding='utf-8') as f:
    schema_sql = f.read()

# Ejecutar script SQL
cursor.executescript(schema_sql)
conn.commit()

print('☑ Esquema de base de datos creado')

# Verificar tablas creadas
cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
tables = cursor.fetchall()

print(f'\n📋 TABLAS CREADAS:')
for table in tables:
    print(f'    - {table[0]}')

# Verificar vistas
cursor.execute("SELECT name FROM sqlite_master WHERE type='view'")
views = cursor.fetchall()

print(f'\n🕒 VISTAS CREADAS:')
for view in views:
    print(f'    - {view[0]}')
```

¿Qué hace `.executescript()` ?

- Ejecuta múltiples sentencias SQL separadas por `;`
- Útil para scripts grandes como schema.sql

Salida:

```
☑ Esquema de base de datos creado

📋 TABLAS CREADAS:
- customers
- products
- locations
- sales

🕒 VISTAS CREADAS:
- vw_sales_complete
```




PASO 3: CARGAR DATOS A LA BASE DE DATOS

► 3.1 Cargar tabla customers

Nueva celda:

```
print('\n' + '=' * 80)
print('CARGANDO DATOS A LA BASE DE DATOS')
print('=' * 80)

# Preparar datos de clientes (únicos)
df_customers_unique = df_merged[['customer_id', 'age', 'gender', 'location', 'membership_years']]

print(f'\n📊 TABLA: customers')
print(f'Registros a insertar: {len(df_customers_unique):,}')

# Insertar datos
df_customers_unique.to_sql('customers', conn, if_exists='append', index=False)

# Verificar inserción
cursor.execute('SELECT COUNT(*) FROM customers')
count = cursor.fetchone()[0]
print(f'✅ Registros insertados: {count:,}')
```

Método `.to_sql()` :

Parámetro	Descripción
<code>'tabla'</code>	Nombre de la tabla destino
<code>conn</code>	Conexión a la base de datos
<code>if_exists='append'</code>	Agregar a tabla existente
<code>index=False</code>	No incluir índice de DataFrame

Opciones de `if_exists` :

- `'fail'` : Error si tabla existe (por defecto)
- `'replace'` : Eliminar y recrear tabla
- `'append'` : Agregar registros a tabla existente

Salida:

```
=====
CARGANDO DATOS A LA BASE DE DATOS
=====

📊 TABLA: customers
Registros a insertar: 4,913
✅ Registros insertados: 4,913
```

► 3.2 Cargar tabla products

Nueva celda:

```
# Preparar datos de productos
product_categories = df_merged['product_category'].unique()
df_products = pd.DataFrame({
    'product_category': product_categories,
    'description': [
        'Ropa y vestimenta',
        'Dispositivos electrónicos',
        'Calzado deportivo y casual',
        'Libros físicos y digitales',
        'Productos de belleza',
        'Juguetes y entretenimiento'
    ]
})

print(f'\n📊 TABLA: products')
print(f'Registros a insertar: {len(df_products)}')

# Insertar datos
df_products.to_sql('products', conn, if_exists='append', index=False)

# Verificar
cursor.execute('SELECT COUNT(*) FROM products')
count = cursor.fetchone()[0]
print(f'✅ Registros insertados: {count}')
```

Salida:

```
📊 TABLA: products
Registros a insertar: 6
✅ Registros insertados: 6
```

► 3.3 Cargar tabla locations

Nueva celda:

```
# Preparar datos de ubicaciones
locations_unique = df_merged['location'].unique()
df_locations = pd.DataFrame({
    'location': locations_unique,
    'city': locations_unique, # En este caso, location ya es el nombre de ciudad
    'country': ['Turkey'] * len(locations_unique)
})

print(f'\n📊 TABLA: locations')
print(f'Registros a insertar: {len(df_locations)}')

# Insertar datos
df_locations.to_sql('locations', conn, if_exists='append', index=False)

# Verificar
cursor.execute('SELECT COUNT(*) FROM locations')
count = cursor.fetchone()[0]
print(f'✅ Registros insertados: {count}')
```

Salida:

```
📊 TABLA: locations
Registros a insertar: 3
✅ Registros insertados: 3
```

► 3.4 Cargar tabla sales

Nueva celda:

```
# Preparar datos de ventas
df_sales_to_db = df_merged[[
    'transaction_id', 'customer_id', 'invoice_date', 'invoice_no',
    'product_category', 'quantity', 'price', 'payment_method', 'shopping_mall'
]].copy()

# Convertir fecha a formato string compatible con SQLite
df_sales_to_db['invoice_date'] = df_sales_to_db['invoice_date'].dt.strftime('%Y-%m-%d')

print(f'\n📊 TABLA: sales')
print(f'Registros a insertar: {len(df_sales_to_db):,}')

# Insertar en lotes para mejor rendimiento
batch_size = 10000
for i in range(0, len(df_sales_to_db), batch_size):
    batch = df_sales_to_db.iloc[i:i+batch_size]
    batch.to_sql('sales', conn, if_exists='append', index=False)
    print(f'    Lote {i//batch_size + 1}: {len(batch):,} registros')

# Verificar
cursor.execute('SELECT COUNT(*) FROM sales')
count = cursor.fetchone()[0]
print(f'✅ Total de registros insertados: {count:,}')

# Confirmar todos los cambios
conn.commit()
print(f'\n✅ TODOS LOS DATOS CARGADOS A LA BASE DE DATOS')
```

¿Por qué insertar en lotes?

- Mejor rendimiento para grandes volúmenes
- Reduce uso de memoria
- Permite seguimiento de progreso

Salida:

 TABLA: sales

Registros a insertar: 99,338

Lote 1: 10,000 registros

Lote 2: 10,000 registros

Lote 3: 10,000 registros

Lote 4: 10,000 registros

Lote 5: 10,000 registros

Lote 6: 10,000 registros

Lote 7: 10,000 registros

Lote 8: 10,000 registros

Lote 9: 10,000 registros

Lote 10: 9,338 registros

☒ Total de registros insertados: 99,338

☒ TODOS LOS DATOS CARGADOS A LA BASE DE DATOS



PASO 4: CONSULTAS SQL DE ANÁLISIS

► 4.1 Crear archivo consultas.sql

Archivo: `sql/consultas.sql`

```
-- =====
-- CONSULTAS ANALÍTICAS SQL
-- Proyecto: ETL Entidad Financiera
-- =====

-- -----
-- CONSULTA 1: Método de pago más utilizado (Item 4a)
-- -----

SELECT
    payment_method AS 'Método de Pago',
    COUNT(*) AS 'Total Transacciones',
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM sales), 2) AS 'Porcentaje (%)'
FROM sales
GROUP BY payment_method
ORDER BY COUNT(*) DESC;

-- -----
-- CONSULTA 2: Género con más compras (Item 4b)
-- -----

SELECT
    c.gender AS 'Género',
    COUNT(s.transaction_id) AS 'Total Compras',
    ROUND(COUNT(s.transaction_id) * 100.0 / (SELECT COUNT(*) FROM sales), 2) AS 'Porcentaje (%)'
FROM sales s
INNER JOIN customers c ON s.customer_id = c.customer_id
GROUP BY c.gender
ORDER BY COUNT(s.transaction_id) DESC;

-- -----
-- CONSULTA 3: Categoría con mayor facturación (Item 4c)
-- -----

SELECT
    product_category AS 'Categoría',
    COUNT(*) AS 'Transacciones',
    SUM(total_sale) AS 'Facturación Total',
    ROUND(AVG(total_sale), 2) AS 'Venta Promedio',
    ROUND(SUM(total_sale) * 100.0 / (SELECT SUM(total_sale) FROM sales), 2) AS 'Porcentaje (%)'
FROM sales
GROUP BY product_category
ORDER BY SUM(total_sale) DESC;

-- -----
-- CONSULTA 4: Ticket promedio por ubicación (Item 4d)
-- -----

SELECT
```

```
c.location AS 'Ubicación',
COUNT(s.transaction_id) AS 'Transacciones',
ROUND(AVG(s.total_sale), 2) AS 'Ticket Promedio',
ROUND(MIN(s.total_sale), 2) AS 'Venta Mínima',
ROUND(MAX(s.total_sale), 2) AS 'Venta Máxima'
FROM sales s
INNER JOIN customers c ON s.customer_id = c.customer_id
GROUP BY c.location
ORDER BY AVG(s.total_sale) DESC;
```

```
-- -----
-- CONSULTA 5: Precio promedio por categoría (Item 4e)
-- -----
```

```
SELECT
    product_category AS 'Categoría',
    COUNT(*) AS 'Productos Vendidos',
    ROUND(AVG(price), 2) AS 'Precio Promedio',
    ROUND(MIN(price), 2) AS 'Precio Mínimo',
    ROUND(MAX(price), 2) AS 'Precio Máximo'
FROM sales
GROUP BY product_category
ORDER BY AVG(price) DESC;
```

```
-- -----
-- CONSULTA 6: Ventas mensuales
-- -----
```

```
SELECT
    strftime('%Y-%m', invoice_date) AS 'Mes',
    COUNT(*) AS 'Transacciones',
    SUM(total_sale) AS 'Ventas Totales',
    ROUND(AVG(total_sale), 2) AS 'Ticket Promedio'
FROM sales
GROUP BY strftime('%Y-%m', invoice_date)
ORDER BY Mes;
```

```
-- -----
-- CONSULTA 7: Top 10 clientes por facturación
-- -----
```

```
SELECT
    c.customer_id AS 'ID Cliente',
    c.age AS 'Edad',
    c.gender AS 'Género',
    c.location AS 'Ubicación',
    COUNT(s.transaction_id) AS 'Compras',
    SUM(s.total_sale) AS 'Facturación Total',
    ROUND(AVG(s.total_sale), 2) AS 'Ticket Promedio'
```



```
FROM customers c
INNER JOIN sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id
ORDER BY SUM(s.total_sale) DESC
LIMIT 10;
```

```
-- -----
-- CONSULTA 8: Ventas por día de la semana
-- -----
```

```
SELECT
    CASE CAST(strftime('%w', invoice_date) AS INTEGER)
        WHEN 0 THEN 'Domingo'
        WHEN 1 THEN 'Lunes'
        WHEN 2 THEN 'Martes'
        WHEN 3 THEN 'Miércoles'
        WHEN 4 THEN 'Jueves'
        WHEN 5 THEN 'Viernes'
        WHEN 6 THEN 'Sábado'
    END AS 'Día de la Semana',
    COUNT(*) AS 'Transacciones',
    SUM(total_sale) AS 'Ventas Totales',
    ROUND(AVG(total_sale), 2) AS 'Ticket Promedio'
FROM sales
GROUP BY strftime('%w', invoice_date)
ORDER BY strftime('%w', invoice_date);
```

```
-- -----
-- CONSULTA 9: Productos más vendidos por cantidad
-- -----
```

```
SELECT
    product_category AS 'Categoría',
    SUM(quantity) AS 'Unidades Vendidas',
    COUNT(*) AS 'Transacciones',
    ROUND(AVG(quantity), 2) AS 'Cantidad Promedio'
FROM sales
GROUP BY product_category
ORDER BY SUM(quantity) DESC;
```

```
-- -----
-- CONSULTA 10: Análisis de membresía de clientes
-- -----
```

```
SELECT
    c.membership_years AS 'Años de Membresía',
    COUNT(DISTINCT c.customer_id) AS 'Clientes',
    COUNT(s.transaction_id) AS 'Transacciones',
    SUM(s.total_sale) AS 'Facturación Total',
```

```

        ROUND(AVG(s.total_sale), 2) AS 'Ticket Promedio'
FROM customers c
INNER JOIN sales s ON c.customer_id = s.customer_id
GROUP BY c.membership_years
ORDER BY c.membership_years;

```

```

-- -----
-- CONSULTA 11: Comparación de métodos de pago por categoría
-- -----

```

```

SELECT
    product_category AS 'Categoría',
    payment_method AS 'Método de Pago',
    COUNT(*) AS 'Transacciones',
    SUM(total_sale) AS 'Ventas Totales'
FROM sales
GROUP BY product_category, payment_method
ORDER BY product_category, SUM(total_sale) DESC;

```

```

-- -----
-- CONSULTA 12: Análisis de rangos de edad
-- -----

```

```

SELECT
    CASE
        WHEN c.age < 30 THEN 'Joven (18-29)'
        WHEN c.age < 50 THEN 'Adulto (30-49)'
        ELSE 'Senior (50+)'
    END AS 'Grupo de Edad',
    COUNT(DISTINCT c.customer_id) AS 'Clientes',
    COUNT(s.transaction_id) AS 'Transacciones',
    SUM(s.total_sale) AS 'Facturación Total',
    ROUND(AVG(s.total_sale), 2) AS 'Ticket Promedio'
FROM customers c
INNER JOIN sales s ON c.customer_id = s.customer_id
GROUP BY
    CASE
        WHEN c.age < 30 THEN 'Joven (18-29)'
        WHEN c.age < 50 THEN 'Adulto (30-49)'
        ELSE 'Senior (50+)'
    END
ORDER BY SUM(s.total_sale) DESC;

```

```

-- -----
-- CONSULTA 13: Centros comerciales con mayor facturación
-- -----

```

```

SELECT
    shopping_mall AS 'Centro Comercial',

```

```

        COUNT(*) AS 'Transacciones',
        SUM(total_sale) AS 'Facturación Total',
        ROUND(AVG(total_sale), 2) AS 'Ticket Promedio'
FROM sales
GROUP BY shopping_mall
ORDER BY SUM(total_sale) DESC;

-- -----
-- CONSULTA 14: Ventas por trimestre
-- -----
SELECT
    strftime('%Y', invoice_date) AS 'Año',
    CASE
        WHEN CAST(strftime('%m', invoice_date) AS INTEGER) BETWEEN 1 AND 3 THEN 'Q1'
        WHEN CAST(strftime('%m', invoice_date) AS INTEGER) BETWEEN 4 AND 6 THEN 'Q2'
        WHEN CAST(strftime('%m', invoice_date) AS INTEGER) BETWEEN 7 AND 9 THEN 'Q3'
        ELSE 'Q4'
    END AS 'Trimestre',
    COUNT(*) AS 'Transacciones',
    SUM(total_sale) AS 'Ventas Totales'
FROM sales
GROUP BY Año, Trimestre
ORDER BY Año, Trimestre;

-- -----
-- CONSULTA 15: Clientes sin compras recientes (últimos 6 meses)
-- -----
SELECT
    c.customer_id AS 'ID Cliente',
    c.age AS 'Edad',
    c.gender AS 'Género',
    c.location AS 'Ubicación',
    MAX(s.invoice_date) AS 'Última Compra',
    COUNT(s.transaction_id) AS 'Total Compras Históricas'
FROM customers c
INNER JOIN sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id
HAVING MAX(s.invoice_date) < date((SELECT MAX(invoice_date) FROM sales), '-6 months')
ORDER BY MAX(s.invoice_date) ASC
LIMIT 20;

-- -----
-- CONSULTA 16: Análisis de lealtad (compras repetidas)
-- -----
SELECT
    CASE

```

```

        WHEN compras = 1 THEN '1 compra'
        WHEN compras BETWEEN 2 AND 5 THEN '2-5 compras'
        WHEN compras BETWEEN 6 AND 10 THEN '6-10 compras'
        WHEN compras BETWEEN 11 AND 20 THEN '11-20 compras'
        ELSE '20+ compras'
    END AS 'Frecuencia de Compra',
    COUNT(*) AS 'Clientes'
FROM (
    SELECT customer_id, COUNT(*) as compras
    FROM sales
    GROUP BY customer_id
) AS customer_purchases
GROUP BY
    CASE
        WHEN compras = 1 THEN '1 compra'
        WHEN compras BETWEEN 2 AND 5 THEN '2-5 compras'
        WHEN compras BETWEEN 6 AND 10 THEN '6-10 compras'
        WHEN compras BETWEEN 11 AND 20 THEN '11-20 compras'
        ELSE '20+ compras'
    END;

```

```

-- -----
-- CONSULTA 17: Uso de la vista vw_sales_complete
-- -----

SELECT
    location AS 'Ubicación',
    gender AS 'Género',
    COUNT(*) AS 'Transacciones',
    SUM(total_sale) AS 'Facturación'
FROM vw_sales_complete
GROUP BY location, gender
ORDER BY location, SUM(total_sale) DESC;

-- =====
-- FIN DE CONSULTAS
-- =====

```



PASO 5: EJECUTAR CONSULTAS DESDE PYTHON

► 5.1 Ejecutar consulta SQL y mostrar resultados

Nueva celda:

```
print('\n' + '=' * 80)
print('EJECUTANDO CONSULTAS SQL')
print('=' * 80)

# CONSULTA 1: Método de pago más utilizado
print('\n📊 CONSULTA 1: Método de pago más utilizado')
print('-' * 80)

query = """
SELECT
    payment_method AS 'Método de Pago',
    COUNT(*) AS 'Total Transacciones',
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM sales), 2) AS 'Porcentaje (%)'
FROM sales
GROUP BY payment_method
ORDER BY COUNT(*) DESC;
"""

df_result = pd.read_sql_query(query, conn)
print(df_result.to_string(index=False))
```

¿Qué hace `pd.read_sql_query()` ?

- Ejecuta consulta SQL
- Retorna resultados como DataFrame de Pandas
- Permite usar todos los métodos de Pandas sobre resultados SQL

Salida:

```
=====
EJECUTANDO CONSULTAS SQL
=====
```

```
 CONSULTA 1: Método de pago más utilizado
```

```
-----
Método de Pago  Total Transacciones  Porcentaje (%)
      Cash                44429                44.72
Credit Card      36366                36.61
Debit Card        18543                18.67
```

► 5.2 Función para ejecutar consultas múltiples

Nueva celda:

```
def ejecutar_consulta(titulo, query, conn):
    """Ejecuta consulta SQL y muestra resultados formateados"""
    print(f'\n📊 {titulo}')
    print('-' * 80)

    df_result = pd.read_sql_query(query, conn)

    if len(df_result) == 0:
        print('⚠️ No se encontraron resultados')
    else:
        print(df_result.to_string(index=False))
        print(f'\n✅ {len(df_result)} registros retornados')

    return df_result

# Ejemplo de uso con Consulta 3
query_3 = """
SELECT
    product_category AS 'Categoría',
    COUNT(*) AS 'Transacciones',
    SUM(total_sale) AS 'Facturación Total',
    ROUND(AVG(total_sale), 2) AS 'Venta Promedio'
FROM sales
GROUP BY product_category
ORDER BY SUM(total_sale) DESC;
"""

df_cat = ejecutar_consulta('CONSULTA 3: Categoría con mayor facturación', query_3, conn)
```

Salida:

 CONSULTA 3: Categoría con mayor facturación

```
-----
Categoría  Transacciones  Facturación Total  Venta Promedio
Clothing   45380           113802356.68     2508.16
Technology 23804           75137542.03       3157.35
Shoes      19833           50036791.35       2522.44
Books      19963           49866729.73       2499.00
Cosmetics  19945           24931486.48       1250.16
Toys       19951           24932282.18       1249.92
```

✅ 6 registros retornados

► 5.3 Ejecutar todas las consultas del archivo

Nueva celda:

```
# Leer todas las consultas del archivo
with open('../sql/consultas.sql', 'r', encoding='utf-8') as f:
    sql_content = f.read()

# Separar consultas individuales (por comentario de título)
import re
consultas = re.split(r'-- -{70,}', sql_content)

# Filtrar solo consultas con SELECT
consultas_sql = [q.strip() for q in consultas if 'SELECT' in q]

print(f'✅ {len(consultas_sql)} consultas encontradas en consultas.sql')
print('\nEjecutando primeras 5 consultas...\n')

# Ejecutar primeras 5
for i, query in enumerate(consultas_sql[:5], 1):
    # Extraer título del comentario
    titulo_match = re.search(r'CONSULTA \d+: (.+)', query)
    titulo = titulo_match.group(0) if titulo_match else f'Consulta {i}'

    # Extraer solo la parte SELECT
    query_clean = query[query.find('SELECT'):]

    try:
        df = ejecutar_consulta(titulo, query_clean, conn)
    except Exception as e:
        print(f'❌ Error en {titulo}: {e}')
```



PASO 6: CERRAR CONEXIÓN

Nueva celda:


```
# Cerrar conexión a la base de datos
conn.close()

print('\n' + '=' * 80)
print('☑ CONEXIÓN A BASE DE DATOS CERRADA')
print('=' * 80)
print(f'\nBase de datos guardada en: {db_path}')
print(f'Tamaño del archivo: {os.path.getsize(db_path) / (1024*1024):.2f} MB')
```

Salida:

```
=====
☑ CONEXIÓN A BASE DE DATOS CERRADA
=====
```

```
Base de datos guardada en: ../sql/financial_analysis.db
Tamaño del archivo: 12.34 MB
```



PASO 7: COMMIT A GIT

```
# Guardar archivos SQL y base de datos
git add sql/schema.sql
git add sql/consultas.sql
git add sql/financial_analysis.db
git add notebooks/analisis_etl.ipynb
git commit -m "Implementar base de datos SQLite con 17 consultas analíticas"
git push origin main
```



RESUMEN SQL

► Archivos creados:

Archivo	Descripción	Líneas
<code>schema.sql</code>	Definición de tablas, índices y vistas	120+
<code>consultas.sql</code>	17 consultas analíticas SQL	300+
<code>financial_analysis.db</code>	Base de datos SQLite con todos los datos	12 MB

► Tablas en la base de datos:

Tabla	Registros	Descripción
<code>customers</code>	4,913	Información de clientes
<code>products</code>	6	Catálogo de productos
<code>locations</code>	3	Ubicaciones de tiendas
<code>sales</code>	99,338	Transacciones de venta

► Consultas SQL principales:

1. Método de pago más utilizado (Item 4a)
2. Género con más compras (Item 4b)
3. Categoría con mayor facturación (Item 4c)
4. Ticket promedio por ubicación (Item 4d)
5. Precio promedio por categoría (Item 4e)
- 6-17. Análisis adicionales (temporales, clientes, lealtad, etc.)



COMANDOS SQL APRENDIDOS

Comando	Función
<code>CREATE TABLE</code>	Crear tabla
<code>PRIMARY KEY</code>	Clave primaria
<code>FOREIGN KEY</code>	Clave foránea (relación)
<code>CHECK</code>	Constraint de validación
<code>INDEX</code>	Índice para optimizar búsquedas
<code>CREATE VIEW</code>	Vista (consulta guardada)
<code>SELECT ... FROM</code>	Consultar datos
<code>GROUP BY</code>	Agrupar por columna
<code>ORDER BY</code>	Ordenar resultados
<code>JOIN</code>	Combinar tablas
<code>COUNT()</code> , <code>SUM()</code> , <code>AVG()</code>	Funciones de agregación
<code>ROUND()</code>	Redondear números
<code>strftime()</code>	Formatear fechas



PRÓXIMOS PASOS

Ver **MÓDULO 6: INTEGRACIÓN GITHUB Y COLAB** para:

- Configurar repositorio en GitHub
 - Crear badge de Colab
 - Configurar clonado automático
 - Probar ejecución en Colab
-

Documento creado: 8 de noviembre de 2025

Parte de: Documentación Técnica Completa - Proyecto ETL TSCDIA